Yarn Holds It All Together

I find it a little hard to get my head around what some of these JavaScript tools are supposed to be doing, in part because they can be a little abstract. Instead, I find it easier to discuss what problems esbuild and its related tools are solving for us. As far as our Rails app is concerned, we're juggling two external pieces of software that solve related problems for our client-side code: Yarn and esbuild.

The problem Yarn is designed to solve is: "What version of all my dependent modules does my program rely on?" Yarn is for JavaScript and npm modules what Bundler is for Ruby gems. All the JavaScript packaging tools supported by jsbundling—esbuild, rollup, and webpack—use Yarn and the package.json file to manage dependencies. (As I'm sure I'll be saying a lot in this chapter, the importmap-rails tool works differently.)

Yarn's role in life is to make sure we have a manifest of the exact version of all our dependent modules and all their dependent modules and so on. Because JavaScript tends to have more but smaller dependencies than Ruby, managing all of them can be quite a load. Like Bundler, Yarn maintains a lock file with the current versions of all the modules in use. Unlike Bundler, your dependent modules are stored with the application itself, in a subdirectory called node_modules. Also unlike Bundler, if two modules are dependent on different versions of the same underlying module, Yarn will bring in both versions and allow each module to access its own version.

Yarn manages dependencies through a file named package.json—it uses the same format for package.json as Node Package Manager (npm) uses. (I'm not going to go over the fine details of installing Yarn on your system; you can find full documentation on the official website.[52] I'll only say that for the moment, Yarn 2.0 is not fully supported.)

Let's take a look at our code to see what a package.json file looks like at this point (again, this is edited because you can't put newlines in the script commands in a